

APOSTILA

Linguagens Formais e Teoria dos Autômatos

*Dos alfabetos finitos às Máquinas de Turing:
fundamentos, modelos e aplicações em computação*

Material de estudo introdutório
Ciência da Computação e Engenharia de Computação

Sumário

1. Introdução aos Conceitos Fundamentais	2
2. A Hierarquia de Chomsky	5
3. Autômatos Finitos e Linguagens Regulares	7
4. Gramáticas e Linguagens Livres de Contexto	11
5. Máquinas de Turing e os Limites da Computação	14
6. Aplicações Práticas na Computação	16
7. Referências e Leituras Complementares	17

1. Introdução aos Conceitos Fundamentais

A Teoria das Linguagens Formais e Autômatos é o ramo da Ciência da Computação que estuda, de forma matemática e rigorosa, como conjuntos de cadeias de símbolos podem ser descritos e reconhecidos por máquinas abstratas. Ela é a base teórica sobre a qual se constroem compiladores, interpretadores, motores de expressões regulares, analisadores de protocolos de rede e, em última instância, a própria noção do que significa "computar" algo.

Antes de falar de autômatos, precisamos formalizar alguns objetos matemáticos básicos que serão usados ao longo de toda a apostila.

1.1 Alfabeto

Um **alfabeto**, denotado por Σ (sigma), é um conjunto finito e não vazio de símbolos (também chamados de *tokens* ou *caracteres*).

Exemplos de alfabetos

$\Sigma_1 = \{0, 1\} \rightarrow$ alfabeto binário

$\Sigma_2 = \{a, b, c, \dots, z\} \rightarrow$ alfabeto do português (sem acentos)

$\Sigma_3 = \{if, else, while, +, -, (,)\} \rightarrow$ tokens de uma linguagem de programação simplificada

1.2 Cadeia (String)

Uma **cadeia** (ou *string*) é uma sequência finita de símbolos retirados de um alfabeto Σ . O tamanho de uma cadeia w , denotado $|w|$, é o número de símbolos que a compõem. A **cadeia vazia**, denotada por ϵ (épsilon), é a cadeia com zero símbolos, ou seja, $|\epsilon| = 0$.

Operações sobre cadeias

- Concatenação: se $w = "ab"$ e $v = "cd"$, então $wv = "abcd"$.
- Potência: w^n significa w concatenada consigo mesma n vezes ($w^0 = \epsilon$).
- Reverso: w^R inverte a ordem dos símbolos; se $w = "abc"$, então $w^R = "cba"$.
- Prefixo e sufixo: $"ab"$ é prefixo de $"abc"$; $"bc"$ é sufixo de $"abc"$.

1.3 Linguagem Formal

O conjunto de todas as cadeias possíveis (incluindo ϵ) que podem ser formadas a partir de um alfabeto Σ é denotado por Σ^* (fecho de Kleene). Uma **linguagem formal** L é simplesmente qualquer subconjunto de Σ^* , ou seja, $L \subseteq \Sigma^*$.

Exemplo

Se $\Sigma = \{0, 1\}$, então $\Sigma^* = \{\epsilon, 0, 1, 00, 01, 10, 11, 000, \dots\}$

A linguagem $L = \{w \in \Sigma^* \mid w \text{ tem um número par de 1's}\}$ é um subconjunto específico de Σ^* — por exemplo, "0", "11", "1001" e "101" pertencem a L , mas "1", "10" e "111" não pertencem.

Note que uma linguagem pode ser finita (como $\{00, 01, 10, 11\}$) ou infinita (como "todas as cadeias binárias com número par de 1's"). O grande problema estudado por essa teoria é: dado um critério que define uma linguagem, existe uma máquina capaz de decidir mecanicamente se uma cadeia pertence ou não a essa linguagem? E, se existir, qual é a máquina mais simples capaz de fazer isso?

1.4 Operações sobre Linguagens

Como linguagens são conjuntos, podemos aplicar operações de teoria dos conjuntos (união, interseção, complemento, diferença), além de operações específicas:

- União: $L_1 \cup L_2 = \{w \mid w \in L_1 \text{ ou } w \in L_2\}$
- Concatenação: $L_1 L_2 = \{xy \mid x \in L_1 \text{ e } y \in L_2\}$
- Fecho de Kleene: $L^* = \text{união de } L^n \text{ para todo } n \geq 0 \text{ (zero ou mais concatenações de } L)$
- Fecho positivo: $L^+ = \text{união de } L^n \text{ para } n \geq 1 \text{ (uma ou mais concatenações, exclui } \epsilon \text{ a menos que } \epsilon \in L)$

Essas operações são a base para a construção de expressões regulares, que veremos no Capítulo 3.

2. A Hierarquia de Chomsky

Em 1956, o linguista Noam Chomsky propôs uma classificação de gramáticas formais em quatro níveis, organizados por seu poder expressivo. Essa hierarquia é o mapa mental mais importante de toda a disciplina: ela relaciona diretamente um tipo de gramática (um mecanismo de geração de cadeias) com um tipo de autômato (um mecanismo de reconhecimento de cadeias) e com uma classe de linguagens.

2.1 Os quatro níveis

Tipo	Gramática	Linguagem	Autômato reconhecedor	Memória do autômato
Tipo 3	Regular	Regular	Autômato Finito (AFD/AFN)	Nenhuma (só o estado)
Tipo 2	Livre de Contexto	Livre de Contexto	Autômato de Pilha (PDA)	Pilha (LIFO)
Tipo 1	Sensível ao Contexto	Sensível ao Contexto	Aut. Linearmente Limitado	Fita limitada
Tipo 0	Irrestrita	Recursivamente Enumerável	Máquina de Turing	Fita infinita

Um ponto crucial dessa hierarquia é que ela forma uma cadeia de inclusões estritas:

Regulares $\subset$ Livres de Contexto $\subset$ Sensíveis ao Contexto $\subset$ Recursivamente Enumeráveis

Isso significa que toda linguagem regular também é livre de contexto (mas não o contrário), toda linguagem livre de contexto também é sensível ao contexto, e assim por diante. Quanto mais "acima" na hierarquia (Tipo 0), mais expressiva é a gramática — mas, em compensação, mais difícil (ou até impossível) é construir um reconhecedor eficiente para ela.

2.2 Gramáticas formais: definição geral

Formalmente, uma gramática é uma tupla $G = (V, \Sigma, R, S)$, onde:

- V é um conjunto finito de símbolos não-terminais (variáveis);
- Σ é o alfabeto de símbolos terminais ($\Sigma \cap V = \emptyset$);
- R é um conjunto finito de regras de produção, da forma $\alpha \rightarrow \beta$;
- $S \in V$ é o símbolo inicial.

O que diferencia os quatro tipos da hierarquia é justamente a forma permitida para as regras em R :

Tipo	Restrição sobre as regras $\alpha \rightarrow \beta$
Regular (3)	$A \rightarrow aB$, $A \rightarrow a$ ou $A \rightarrow \epsilon$ (A, B não-terminais; a terminal)
Livre de contexto (2)	$A \rightarrow \beta$, onde A é um único não-terminal e β é qualquer cadeia
Sensível ao contexto (1)	$\alpha A \beta \rightarrow \alpha \gamma \beta$, com γ não vazio ($ \alpha \beta < \alpha \gamma \beta $)
Irrestrita (0)	$\alpha \rightarrow \beta$, sem nenhuma restrição estrutural

Na prática, os dois tipos mais usados no dia a dia da programação são as linguagens regulares (Tipo 3) e as linguagens livres de contexto (Tipo 2) — os próximos capítulos são dedicados quase inteiramente a elas, além de uma introdução ao Tipo 0 (Máquinas de Turing), essencial para entender os limites da computação.

3. Autômatos Finitos e Linguagens Regulares

Os autômatos finitos são os modelos computacionais mais simples da hierarquia, mas também os mais usados na prática cotidiana — toda vez que você usa uma expressão regular para validar um e-mail, ou o analisador léxico de um compilador reconhece um token, um autômato finito (ou algo equivalente a ele) está atuando por trás dos panos.

3.1 Definição formal

Um **Autômato Finito Determinístico (AFD)** é definido formalmente como uma quintupla:

$$M = (Q, \Sigma, \delta, q_0, F)$$

- Q : conjunto finito de estados;
- Σ : alfabeto de entrada;
- $\delta: Q \times \Sigma \rightarrow Q$, a função de transição (determinística: um único destino para cada par estado/símbolo);
- $q_0 \in Q$: estado inicial;
- $F \subseteq Q$: conjunto de estados finais (de aceitação).

Uma cadeia w é aceita pelo autômato se, partindo de q_0 e seguindo as transições indicadas por δ para cada símbolo de w , o autômato termina em um estado pertencente a F .

Exemplo prático

Considere um AFD que reconhece a linguagem $L = \{ w \in \{0,1\}^* \mid w \text{ termina em "01"} \}$:

Estados: $Q = \{q_0, q_1, q_2\}$

Estado inicial: q_0

Estados finais: $F = \{q_2\}$

Transições:

$\delta(q_0, 0) = q_1$	$\delta(q_0, 1) = q_0$
$\delta(q_1, 0) = q_1$	$\delta(q_1, 1) = q_2$
$\delta(q_2, 0) = q_1$	$\delta(q_2, 1) = q_0$

Podemos simular a cadeia "1001": $q_0 \xrightarrow{1} q_0 \xrightarrow{0} q_1 \xrightarrow{0} q_1 \xrightarrow{1} q_2$. Como q_2 é final, a cadeia é aceita — de fato, "1001" termina em "01".

3.2 Autômato Finito Não-Determinístico (AFN)

Um **AFN** relaxa a função de transição: δ passa a mapear para um *conjunto* de estados possíveis ($\delta: Q \times \Sigma \rightarrow 2^Q$), permitindo inclusive transições sem consumir símbolo (transições- ϵ). Isso facilita bastante a construção de autômatos a partir de expressões regulares.

Resultado fundamental

Todo AFN pode ser convertido em um AFD equivalente através da construção de subconjuntos (subset construction), na qual cada estado do AFD resultante representa um conjunto de estados possíveis do AFN. Isso prova que AFDs e AFNs reconhecem exatamente a mesma classe de linguagens — as linguagens regulares — apesar do AFN parecer, à primeira vista, mais expressivo.

3.3 Expressões Regulares

As **expressões regulares (regex)** são uma notação algébrica para descrever linguagens regulares, construída a partir de três operações básicas: união ($|$), concatenação e fecho de Kleene ($*$).

Expressão	Linguagem descrita
0^*1	zero ou mais 0's seguidos de exatamente um 1
$(0 1)^*$	todas as cadeias binárias possíveis (equivale a Σ^*)
$(ab)^+$	uma ou mais repetições de "ab"
$a(a b)^*b$	cadeias que começam com 'a' e terminam com 'b'

O **Teorema de Kleene** garante a equivalência total entre expressões regulares e autômatos finitos: toda linguagem descrita por uma expressão regular pode ser reconhecida por um AFN (e, por consequência, por um AFD), e vice-versa. Essa equivalência é o que permite que motores de regex (como os usados em Python, JavaScript, grep, etc.) sejam implementados internamente como autômatos finitos.

3.4 O Pumping Lemma para Linguagens Regulares

Nem toda linguagem é regular. O Pumping Lemma é a principal ferramenta para provar que uma linguagem **NÃO** é regular. Intuitivamente, ele explora o fato de que um autômato finito tem um número limitado de estados — então, para cadeias suficientemente longas, algum estado necessariamente se repete, criando um "laço" que pode ser bombeado (repetido) sem alterar a aceitação.

Enunciado formal

Se L é uma linguagem regular, então existe um número $p \geq 1$ (o "tamanho de bombeamento") tal que toda cadeia $w \in L$ com $|w| \geq p$ pode ser dividida em três partes, $w = xyz$, satisfazendo:

- 1) $|xy| \leq p$
- 2) $|y| > 0$
- 3) para todo $i \geq 0$, a cadeia $x y^i z$ também pertence a L

Exemplo clássico de aplicação: prova-se que $L = \{ a^n b^n \mid n \geq 0 \}$ não é regular, pois qualquer tentativa de dividir uma cadeia $a^p b^p$ em xyz respeitando $|xy| \leq p$ força y a ser composto só de a 's — bombear y (repetir ou remover) quebra o balanceamento entre a 's e b 's, contradizendo a suposição de que L seria regular.

3.5 Minimização de AFDs

Um mesmo AFD pode ter estados redundantes (equivalentes entre si). Existem algoritmos (como o algoritmo de Hopcroft ou o método de partição de estados) que produzem o AFD mínimo equivalente para uma dada linguagem regular — útil tanto para eficiência de execução quanto para comparar se dois autômatos reconhecem a mesma linguagem.

A ideia central do algoritmo é iniciar com uma partição grosseira dos estados (estados finais de um lado, não-finais do outro) e refiná-la iterativamente: dois estados só podem permanecer no mesmo grupo se, para todo símbolo do alfabeto, eles transitarem para grupos equivalentes. Quando nenhuma partição adicional é possível, cada grupo remanescente se torna um único estado no AFD mínimo.

3.6 Fechamento das linguagens regulares

Uma propriedade importante é que a classe das linguagens regulares é fechada sob diversas operações — ou seja, aplicar essas operações a linguagens regulares sempre produz outra linguagem regular:

Operação	Resultado	Ideia da construção
União ($L_1 \cup L_2$)	Regular	AFN com um novo estado inicial ligado por ϵ aos dois autômatos
Concatenação ($L_1 L_2$)	Regular	Liga os estados finais de M_1 ao inicial de M_2 via ϵ
Fecho de Kleene (L^*)	Regular	Adiciona ϵ do novo inicial ao antigo, e dos finais de volta a ele
Complemento ($\Sigma^* - L$)	Regular	Troca estados finais e não-finais em um AFD completo
Interseção ($L_1 \cap L_2$)	Regular	Produto cartesiano dos dois AFDs (autômato produto)

Operação	Resultado	Ideia da construção

Essas construções não são apenas curiosidades teóricas: motores de regex e ferramentas de análise léxica usam exatamente essas ideias para combinar padrões complexos a partir de padrões mais simples.

4. Gramáticas e Linguagens Livres de Contexto

As linguagens regulares são poderosas, mas não conseguem expressar estruturas aninhadas ou recursivas — como parênteses balanceados, blocos de código aninhados, ou a estrutura sintática de expressões aritméticas. Para isso, precisamos subir um nível na hierarquia de Chomsky: as Linguagens Livres de Contexto (LLCs).

4.1 Gramáticas Livres de Contexto (GLC)

Uma **GLC** é uma gramática em que toda regra tem a forma $A \rightarrow \beta$, onde A é um único símbolo não-terminal e β é qualquer combinação de terminais e não-terminais (incluindo ϵ).

Exemplo: expressões aritméticas balanceadas

Gramática para parênteses balanceados:

$$S \rightarrow (S) S \mid \epsilon$$

Gramática para expressões aritméticas simples:

$$E \rightarrow E + T \mid T$$
$$T \rightarrow T * F \mid F$$
$$F \rightarrow (E) \mid id$$

Essa segunda gramática, por exemplo, é a base conceitual de como compiladores entendem precedência de operadores: multiplicação "liga mais forte" que soma porque T (termo) está mais próximo de F (fator) na hierarquia de produções.

4.2 Árvores de derivação e ambiguidade

Cada cadeia derivada por uma GLC pode ser representada por uma árvore de derivação (parse tree), mostrando como os símbolos não-terminais foram expandidos até chegar aos terminais. Uma gramática é dita ambígua quando existe pelo menos uma cadeia com duas árvores de derivação distintas — isso é indesejável em linguagens de programação, pois gera múltiplas interpretações possíveis para o mesmo código.

4.3 Forma Normal de Chomsky (FNC)

Toda GLC pode ser convertida para uma forma normalizada em que cada regra tem exatamente uma das formas:

- $A \rightarrow BC$ (dois não-terminais)
- $A \rightarrow a$ (um único terminal)

Essa normalização é útil tanto para provas formais quanto para algoritmos de parsing como o CYK (Cocke–Younger–Kasami), que decide em tempo $O(n^3)$ se uma cadeia pertence a uma linguagem livre de contexto.

4.4 Autômatos de Pilha (Pushdown Automata — PDA)

O reconhecedor equivalente às GLCs é o **Autômato de Pilha**, que estende o autômato finito com uma memória auxiliar em formato de pilha (LIFO — Last In, First Out), de capacidade ilimitada.

Formalmente, um PDA é definido pela tupla $M = (Q, \Sigma, \Gamma, \delta, q_0, Z_0, F)$, onde Γ é o alfabeto da pilha, Z_0 é o símbolo inicial de pilha, e δ agora também considera (e manipula) o topo da pilha a cada transição.

Intuição

A pilha funciona como uma memória de "pendências": ao ler um símbolo de abertura (como um parêntese "("), o autômato empilha um marcador; ao ler o símbolo de fechamento correspondente ")", ele desempilha. A cadeia é aceita se, ao final da leitura, a pilha estiver vazia (ou em um estado de aceitação configurado) — isso naturalmente captura estruturas aninhadas.

4.5 Limitações: Pumping Lemma para LLCs

Assim como as linguagens regulares, as LLCs também têm um Pumping Lemma análogo — mais complexo, pois envolve dividir a cadeia em cinco partes ($uvxyz$) — usado para provar que certas linguagens exigem mais poder computacional. O exemplo canônico é:

Exemplo clássico

$L = \{ a^n b^n c^n \mid n \geq 0 \}$ não é livre de contexto.

Intuitivamente: uma pilha consegue "casar" dois grupos de símbolos (por exemplo, contar a's ao empilhar e descontar ao ler b's), mas não tem como simultaneamente garantir a correspondência entre três grupos distintos (a's, b's e c's) — faltaria uma segunda pilha.

4.6 Fechamento das linguagens livres de contexto

As LLCs também são fechadas sob união, concatenação e fecho de Kleene — mas, diferentemente das regulares, NÃO são fechadas sob interseção nem complemento em geral. Por exemplo, a interseção de duas linguagens livres de contexto pode resultar em uma linguagem que não é livre de contexto, como acontece com $L = \{ a^n b^n c^m \} \cap \{ a^m b^n c^n \} = \{ a^n b^n c^n \}$, que já vimos não ser livre de contexto.

4.7 Aplicação prática: análise sintática (parsing)

A análise sintática de linguagens de programação é, na prática, o reconhecimento de uma GLC. Os dois grandes paradigmas de parsers são:

- Parsers descendentes (top-down): como LL(1), que constroem a árvore de derivação a partir da raiz (símbolo inicial) em direção às folhas.
- Parsers ascendentes (bottom-up): como LR(0), SLR, LALR (usado pelo Yacc/Bison), que constroem a árvore das folhas em direção à raiz, usando uma pilha explícita — quase uma implementação direta de um PDA.

5. Máquinas de Turing e os Limites da Computação

No topo da Hierarquia de Chomsky está o modelo mais geral de todos: a Máquina de Turing (MT), proposta por Alan Turing em 1936. Ela não foi criada para ser um modelo "prático" de computador, mas sim uma ferramenta matemática para responder a uma pergunta fundamental: o que significa, precisamente, "computar" alguma coisa?

5.1 Definição formal

Uma Máquina de Turing é definida pela tupla $M = (Q, \Sigma, \Gamma, \delta, q_0, q_{\text{aceita}}, q_{\text{rejeita}})$, onde a principal diferença em relação aos modelos anteriores é a presença de uma **fita infinita** (em ambas as direções, ou ao menos ilimitada à direita), com um cabeçote de leitura e escrita que pode se mover para a esquerda ou para a direita a cada passo, além de poder sobrescrever símbolos na fita.

- Γ : alfabeto da fita (inclui Σ e um símbolo especial de "branco");
- $\delta: Q \times \Gamma \rightarrow Q \times \Gamma \times \{E, D\}$, a função de transição — lê um símbolo, escreve um símbolo (possivelmente diferente) e move o cabeçote para a Esquerda ou Direita;
- q_{aceita} e q_{rejeita} : estados de parada especiais.

Essa combinação simples — fita ilimitada, leitura/escrita, movimento em duas direções — é, surpreendentemente, suficiente para expressar qualquer algoritmo que possa ser executado por um computador real (ainda que de forma muito menos eficiente).

5.2 A Tese de Church-Turing

Tese de Church-Turing

Qualquer função que possa ser calculada por um procedimento efetivo (um algoritmo, no sentido intuitivo) pode ser calculada por uma Máquina de Turing.

Essa não é uma afirmação demonstrável matematicamente (pois "procedimento efetivo" é uma noção intuitiva, não formal) — é uma tese amplamente aceita, reforçada pelo fato de que todos os modelos alternativos de computação propostos até hoje (Lambda Cálculo de Church, funções recursivas de Gödel-Kleene, máquinas de registradores, e mesmo computadores reais) provaram ser computacionalmente equivalentes a uma Máquina de Turing.

5.3 Variantes equivalentes

Diversas variantes da Máquina de Turing — fita múltipla, fita bidimensional, não-determinismo — foram propostas, e todas provaram ter exatamente o mesmo poder computacional da MT de fita

única e determinística (embora possam ser mais eficientes em tempo). Isso reforça a robustez do modelo como definição de "computável".

5.4 Linguagens decidíveis e reconhecíveis

Classe	Definição	Comportamento da MT
Decidível (Recursiva)	Existe uma MT que sempre para, aceitando ou rejeitando corretamente	Sempre termina
Recursivamente Enumerável	Existe uma MT que aceita se $w \in L$, mas pode nunca parar se $w \notin L$	Pode não terminar
Não recursivamente enumerável	Nenhuma MT consegue nem mesmo reconhecer a linguagem	N/A

5.5 O Problema da Parada (Halting Problem)

Um dos resultados mais importantes — e surpreendentes — de toda a Ciência da Computação: não existe (e nunca poderá existir) um algoritmo geral capaz de determinar, para qualquer programa P e qualquer entrada w , se P eventualmente para ao processar w .

Prova por diagrama (esboço)

Suponha, por absurdo, que exista uma MT H que decide o problema da parada. Construamos uma MT D que usa H como sub-rotina: $D(P)$ executa $H(P, P)$ e faz o oposto do resultado — se H diz que P para, D entra em loop infinito; se H diz que P não para, D para imediatamente.

Agora execute $D(D)$. Se D para, então H disse que D não para — contradição. Se D não para, então H disse que D para — contradição também. Logo, H não pode existir.

Esse resultado tem consequências práticas diretas: é a razão formal pela qual nenhuma IDE consegue detectar com 100% de certeza todo loop infinito possível em um código arbitrário, e é a base teórica para diversos outros problemas indecidíveis em compiladores e verificação de software.

6. Aplicações Práticas na Computação

Embora a teoria pareça abstrata, cada nível da hierarquia de Chomsky tem uma aplicação direta e cotidiana em engenharia de software.

6.1 Análise léxica (Regular → AFD)

A primeira fase de um compilador, a análise léxica, converte o código-fonte bruto em uma sequência de tokens (palavras-chave, identificadores, operadores, literais). Cada tipo de token é tipicamente definido por uma expressão regular, e ferramentas como Lex/Flex compilam essas expressões em um único AFD eficiente que reconhece todos os tokens simultaneamente.

6.2 Análise sintática (Livre de Contexto → PDA)

A segunda fase, análise sintática, verifica se a sequência de tokens obedece à gramática da linguagem (definida como uma GLC) e constrói a árvore sintática (AST). Ferramentas como Yacc/Bison e ANTLR geram parsers LALR ou LL automaticamente a partir de uma especificação de gramática.

6.3 Expressões regulares no dia a dia

Bibliotecas de regex em Python (módulo `re`), JavaScript, ou ferramentas de linha de comando como `grep` e `sed` são implementações diretas (ou aproximações eficientes) de autômatos finitos, usadas para validação de formulários, busca de padrões em logs, parsing de formatos simples (como datas ou IPs) e processamento de texto em geral.

6.4 Protocolos e sistemas reativos

Máquinas de estado finito também são amplamente usadas para modelar protocolos de comunicação (como o handshake TCP), controladores embarcados, jogos, e fluxos de interface de usuário — qualquer sistema cujo comportamento dependa de um número finito e bem definido de "estados" possíveis.

6.5 Limites computacionais e complexidade

A teoria de Máquinas de Turing é o alicerce da Teoria da Complexidade Computacional (classes P, NP, NP-completo, etc.), que estuda não apenas se um problema é computável, mas quão eficientemente ele pode ser resolvido — um tema central em qualquer área que envolva otimização, criptografia ou algoritmos de larga escala.

7. Referências e Leituras Complementares

Esta apostila tem caráter introdutório e resume os principais conceitos da área. Para um estudo mais aprofundado, com demonstrações completas e exercícios adicionais, recomenda-se consultar bibliografia clássica sobre o tema, entre elas:

- HOPCROFT, J. E.; MOTWANI, R.; ULLMAN, J. D. Introduction to Automata Theory, Languages, and Computation. Pearson.
- SIPSER, M. Introduction to the Theory of Computation. Cengage Learning.
- MENEZES, P. B. Linguagens Formais e Autômatos. Bookman (Editora, em português).
- LEWIS, H. R.; PAPADIMITRIOU, C. H. Elements of the Theory of Computation. Prentice Hall.